

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

SIOPTR: TT9: Week 5

**My Own KERnel (MOKER) version: Assigning a LIFO task
identifier**

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 Adding an LIFO task Identifier

The main idea is to give a specific identifier to LIFO tasks in order to improve the tracing output readability.

2 Adding a new field to LIFO tasks

Open the `linux-6.19.9-moker/kernel/moker/lf_task` file and update it by adding (blue text) an integer attribute called `id`:

```
#ifndef __LIFO_TASK_H_
#define __LIFO_TASK_H_

#include <linux/list.h>

struct sched_lf_entity{
    int id;
    struct list_head node;
};
#endif
```

3 Create a new System call

Open the `linux-6.19.9-moker/kernel/moker/syscalls.h` file and update it by adding (blue text) an integer attribute called `id`:

```
#ifndef __SYSCALLS_H
#define __SYSCALLS_H
int do_moker_tracing (unsigned int enable);

int do_moker_id (int id);
#endif
```

Open the `linux-6.19.9-moker/kernel/moker/syscalls.c` file and update it by adding (blue text) two new functions:

```

#include <linux/syscalls.h>
#include "../sched/sched.h"
#include "trace.h"
...

SYSCALL_DEFINE1(moker_id, int, id)
{
    printk("MOKER: sys_moker_id:[%d] [%d]\n", id, current->pid);
    return do_moker_id(id);
}

int do_moker_id(int id){
    #ifdef CONFIG_MOKER_SCHED_LIFO_POLICY
    printk("MOKER: do_moker_id:[%d] [%d]\n", id, current->pid);
    current->lf.id = -1;
    if(lf_policy(current->policy)){
        current->lf.id = id;
    }
    }
    #endif
}

```

4 Getting a System Call Number

Add a new entry to the system call table. This is located at `linux-6.19-moker/arch/x86/entry/syscalls/syscall_64.tbl`.

The number of the new system call is 436. This system call is used for x86 and x86_64 systems, so the ABI is `common`. The system call name is `moker_tracing` and the entry point is `sys_moker_tracing`.

```

# SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note
#
# 64-bit system call numbers and entry vectors
#
# The format is:
# <number> <abi> <name> <entry point> [<compat entry point> [noreturn]]
#
# The __x64_sys_*() stubs are created on-the-fly for sys_*() system calls
#
# The abi is "common", "64" or "x32" for this file.
#
...
470 common listns    sys_listns
#ifdef CONFIG_MOKER_FRAMEWORK
471 common moker_tracing sys_moker_tracing
472 common moker_id   sys_moker_id
#endif
...

```

5 Update MOKER Tracing

Add a new field to the moker tracing data. For that purpose update `linux-6.19.9-moker/kernel/moker/trace.h` and `linux-6.19.9-moker/kernel/moker/trace.c` files to support such data.

```
#ifndef __TRACE_H_
#define __TRACE_H_

...
struct trace_evt{
    enum evt event;
    unsigned long long time;
    int number;
    pid_t pid;
    int state;
    int prio;
    int policy;
    char comm[TRACE_TASK_COMM_LEN];
    int id;
};

...
#endif
```

```

#include <linux/module.h>
...

#include "../sched/sched.h"

..
static int dequeue (char *buffer){
    int ret = 0, len;
    char evt[10];
    spin_lock(&trace.lock);
    if (!is_empty(trace.read_item, trace.write_item)){ ///if it is not empty

        switch((int)trace.events[trace.read_item].event){
            case MISC_EVT:
                strcpy(evt, "MC_EVT");
                break;
            case SCHED_TICK:
                strcpy(evt, "SCH_TK");
                break;
            case SWITCH_AWAY:
                strcpy(evt, "SWT_AY");
                break;
            case SWITCH_TO:
                strcpy(evt, "SWT_TO");
                break;
            case ENQUEUE_RQ:
                strcpy(evt, "ENQ_RQ");
                break;
            case DEQUEUE_RQ:
                strcpy(evt, "DEQ_RQ");
                break;
            default:
                strcpy(evt, "UK_EVT");
        }

        len = sprintf(buffer, "%llu, ", trace.events[trace.read_item].time);
        len += sprintf(buffer + len, "%s, ", evt);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].id);
        len += sprintf(buffer + len, "%d*, ", (int)trace.events[trace.read_item].number);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].policy);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].prio);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].pid);
        len += sprintf(buffer + len, "%d, ", (int)trace.events[trace.read_item].state);
        len += sprintf(buffer + len, "%s\n", trace.events[trace.read_item].comm);

        increment(&trace.read_item);
        ret = 1;
    }
    spin_unlock(&trace.lock);
    return ret;
}

static int enqueue (enum evt event, unsigned long long time, int number, struct task_struct *p)
{
    int id = -1;
    if (lf_policy(p->policy)){
        id = p->lf.id;
    }
    spin_lock(&trace.lock);
    if (is_full(trace.read_item, trace.write_item))
        increment(&trace.read_item);
    trace.events[trace.write_item].number = number;
    trace.events[trace.write_item].event = event;
    trace.events[trace.write_item].time = time;
    trace.events[trace.write_item].pid = p->pid;
    trace.events[trace.write_item].state = p->_state;
    trace.events[trace.write_item].prio = p->prio;
    trace.events[trace.write_item].policy = p->policy;
    trace.events[trace.write_item].id = id;
    strcpy(trace.events[trace.write_item].comm, p->comm);

    increment(&trace.write_item);
    spin_unlock(&trace.lock);
    return 1;
}
...

```

6 Compiling

Compile the Linux kernel:

```
sudo ./kcompile.sh
```

```
sudo reboot
```

7 Update LIFO tasks

In order to support the new functionality, create a new macro for the new system call number (`defs.h` file).

```
#ifndef DEFS_H_
#define DEFS_H_

...
#define SYS_MOKER_TRACING_ENABLE 471
#define SYS_MOKER_ID 472
...
#endif
```

Update the `task.c` file to invoke the new syscall.

```
#include "defs.h"

...
int main(int argc, char** argv)
{
    ...

    printf("Task(%d,%d): before SCHED_LIFO\n",task_id,getpid());
    param.sched_priority = 0;
    if((sched_setscheduler(0,SCHED_LIFO,&param)) == -1){
        perror("ERROR:sched_setscheduler failed");
        exit(-1);
    }
    printf("Task(%d,%d): after SCHED_LIFO\n",task_id,getpid());

    printf("Task(%d,%d): set ID\n", task_id,getpid());
    if((syscall(SYS_MOKER_ID,task_id)) < 0){
        perror("ERROR: Setting moker id failed");
        exit(-1);
    }

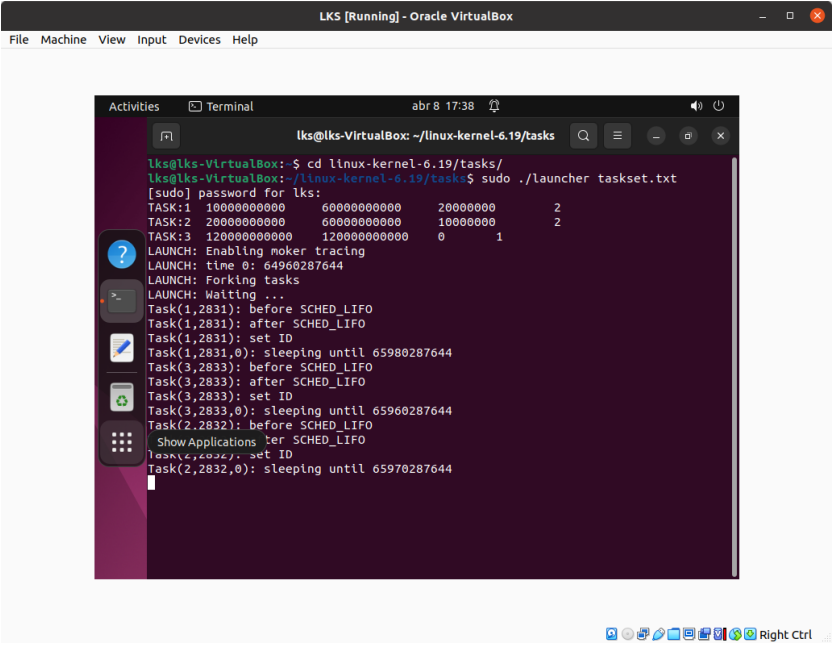
    release = time0 + 0;
    ...
    return 0;
}
```

Compile it by invoking:

`make`

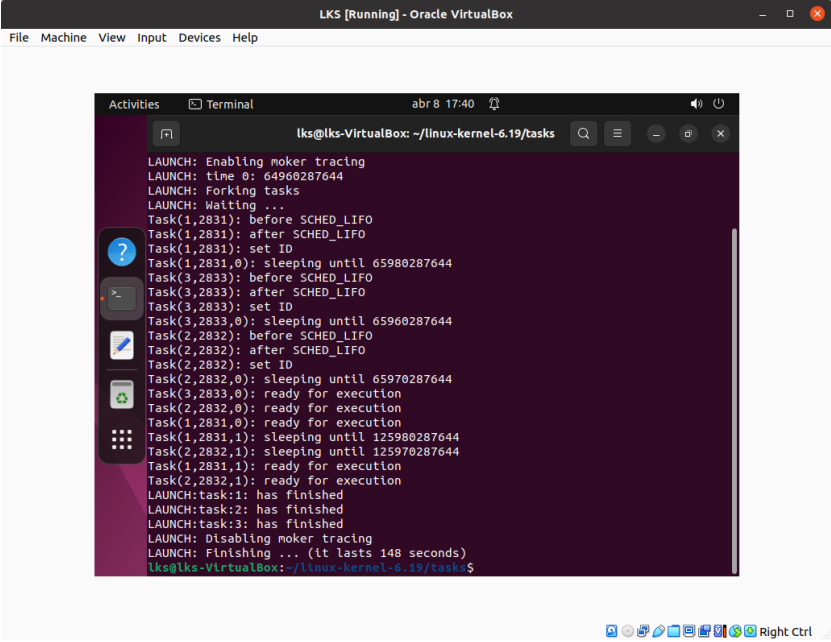
Put the set of tasks running:

`sudo ./launcher taskset.txt`



The screenshot shows a terminal window titled "lks@lks-VirtualBox: ~/linux-kernel-6.19/tasks". The user has executed the command `sudo ./launcher taskset.txt`. The terminal output displays the following information:

```
lks@lks-VirtualBox: ~/linux-kernel-6.19/tasks
lks@lks-VirtualBox:~/linux-kernel-6.19/tasks$ sudo ./launcher taskset.txt
[sudo] password for lks:
TASK:1 10000000000 60000000000 20000000 2
TASK:2 20000000000 60000000000 10000000 2
TASK:3 120000000000 120000000000 0 1
LAUNCH: Enabling moker tracing
LAUNCH: time 0: 64960287644
LAUNCH: Forking tasks
LAUNCH: Waiting ...
Task(1,2831): before SCHED_LIFO
Task(1,2831): after SCHED_LIFO
Task(1,2831): set ID
Task(1,2831,0): sleeping until 65980287644
Task(3,2833): before SCHED_LIFO
Task(3,2833): after SCHED_LIFO
Task(3,2833): set ID
Task(3,2833,0): sleeping until 65960287644
Task(2,2832): before SCHED_LIFO
ShowApplications ter SCHED_LIFO
Task(2,2832,0): set ID
Task(2,2832,0): sleeping until 65970287644
```



```
LKS [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Activities Terminal abr 8 17:40

lks@lks-VirtualBox: ~/linux-kernel-6.19/tasks

LAUNCH: Enabling moker tracing
LAUNCH: time 0: 64960287644
LAUNCH: Forking tasks
LAUNCH: Waiting ...
Task(1,2831): before SCHED_LIFO
Task(1,2831): after SCHED_LIFO
Task(1,2831): set ID
Task(1,2831,0): sleeping until 65980287644
Task(3,2833): before SCHED_LIFO
Task(3,2833): after SCHED_LIFO
Task(3,2833): set ID
Task(3,2833,0): sleeping until 65960287644
Task(2,2832): before SCHED_LIFO
Task(2,2832): after SCHED_LIFO
Task(2,2832): set ID
Task(2,2832,0): sleeping until 65970287644
Task(3,2833,0): ready for execution
Task(2,2832,0): ready for execution
Task(1,2831,0): ready for execution
Task(1,2831,1): sleeping until 125980287644
Task(2,2832,1): sleeping until 125970287644
Task(1,2831,1): ready for execution
Task(2,2832,1): ready for execution
LAUNCH:task:1: has finished
LAUNCH:task:2: has finished
LAUNCH:task:3: has finished
LAUNCH: Disabling moker tracing
LAUNCH: Finishing ... (it lasts 148 seconds)
lks@lks-VirtualBox: ~/linux-kernel-6.19/tasks$
```

Get the scheduling trace:

```
cat /proc/moker_trace > trace.csv
```

Check if the scheduling output is according to the LIFO algorithm.

```
gedit trace.csv
```

```

59961603344,ENQ_RQ,0,*-1*,8,120,2831,0,task
59961612607,DEQ_RQ,1,*-1*,8,120,2831,8193,task
59961613669,SWT_AY,1,*-1*,8,120,2831,8193,task
59961613706,SWT_TO,-1,*-1*,0,120,2833,0,launcher
59961800875,ENQ_RQ,0,*-1*,8,120,2833,0,task
59961807021,DEQ_RQ,3,*-1*,8,120,2833,8193,task
59961807605,SWT_AY,3,*-1*,8,120,2833,8193,task
59961807630,SWT_TO,-1,*-1*,0,120,2832,0,launcher
59961987561,ENQ_RQ,0,*-1*,8,120,2832,0,task
59961993580,DEQ_RQ,2,*-1*,8,120,2832,8193,task
59961994406,SWT_AY,2,*-1*,8,120,2832,8193,task
59961994430,SWT_TO,-1,*-1*,0,120,40,0,kworker/u4:3
65960421870,ENQ_RQ,3,*-1*,8,120,2833,512,task
65960441703,SWT_AY,-1,*-1*,0,120,0,0,swapper/0
65960441974,SWT_TO,3,*-1*,8,120,2833,0,task
65976217895,ENQ_RQ,2,*-1*,8,120,2832,512,task
65976233756,SWT_AY,3,*-1*,8,120,2833,0,task
65976233790,SWT_TO,2,*-1*,8,120,2832,0,task
65991859937,ENQ_RQ,1,*-1*,8,120,2831,512,task
65991862127,SWT_AY,2,*-1*,8,120,2832,0,task
65991862157,SWT_TO,1,*-1*,8,120,2831,0,task
72632031241,DEQ_RQ,1,*-1*,8,120,2831,8193,task
72632032790,SWT_AY,1,*-1*,8,120,2831,8193,task
72632032896,SWT_TO,2,*-1*,8,120,2832,0,task
88807415800,DEQ_RQ,2,*-1*,8,120,2832,8193,task
88807417624,SWT_AY,2,*-1*,8,120,2832,8193,task
88807417667,SWT_TO,3,*-1*,8,120,2833,0,task
125983186197,ENQ_RQ,2,*-1*,8,120,2832,512,task
125983187617,ENQ_RQ,1,*-1*,8,120,2831,512,task
125983193184,SWT_AY,3,*-1*,8,120,2833,0,task
125983193238,SWT_TO,1,*-1*,8,120,2831,0,task
134119968016,DEQ_RQ,1,*-1*,8,120,2831,128,task
134119969175,SWT_AY,1,*-1*,8,120,2831,128,task
134119969256,SWT_TO,2,*-1*,8,120,2832,0,task
150602073909,DEQ_RQ,2,*-1*,8,120,2832,128,task
150602075615,SWT_AY,2,*-1*,8,120,2832,128,task
150602075668,SWT_TO,3,*-1*,8,120,2833,0,task
213344611024,DEQ_RQ,3,*-1*,8,120,2833,128,task
213344613431,SWT_AY,3,*-1*,8,120,2833,128,task
213344613483,SWT_TO,-1,*-1*,2,0,1893,0,rtkit-daemon

```